

RAPPORT TECHNIQUE DE PROJET : PORTAIL ACADÉMIQUE D'INSCRIPTION DISTRIBUÉ (UNIREGIS API)

Présenté par : Abdellahi Ishagh

Encadrant : Dr. EL BENANY Mohamed Mahmoud

Institution : ISB Mauritanie

Stack Principale : Java JDK 21 / Jakarta EE 10 / MicroProfile 6.0 / Docker-Compose

Contexte, Problématique et Objectifs

Le projet **UniRegis API** est né pour pallier les limites critiques des architectures universitaires traditionnelles dites *legacy*, souvent pénalisées par un couplage fort et une gestion d'état lourde (*sessions stateful*).

L'objectif principal est de concevoir un backend moderne, hautement disponible, sécurisé et entièrement **Cloud-Native**, capable de gérer efficacement le flux d'inscription des étudiants de manière distribuée.

2. Architecture Technique End-to-End

Comme le démontre le diagramme d'interaction de notre système (*Application Interaction Flow*), l'application est découpée en couches logiques strictes et modulaires :

- ✓ **Interface Utilisateur (Web UI)** : Développée en HTML/JSP, stylisée avec Tailwind CSS 4, elle communique de manière asynchrone avec le serveur via l'API standard Fetch API.
- ✓ **Couche API REST (JAX-RS Endpoints)** : Point d'entrée unique de la logique métier situé sous le chemin racine `"/api/v1/inscriptions"`.
- ✓ **Couche Métier & Injection de Dépendances (CDI)** : Portée par le composant `InscriptionService` (annoté `@ApplicationScoped`), elle orchestre la validation et s'appuie sur un système d'événements asynchrones (CDI Event Notification) pour déclencher le `NotificationService` (annoté `@Observes`).
- ✓ **Couche de Persistance (JPA & EntityManager)** : Gère les transactions atomiques (`@Transactional`) vers la ressource de données `DataSource java:app/jdbc/uniregis`.

3. Fonctionnalités Clés et Endpoints API

L'application expose des services RESTful pour deux types d'acteurs (Étudiants et Administrateurs):

- **POST /api/v1/inscriptions** : Permet à un utilisateur de créer un dossier. Le système génère automatiquement un identifiant unique standardisé au format **ISB-XXX Matricule Generation**. (Accès Public) .
- **GET /api/v1/inscriptions** : Permet de lister l'ensemble des demandes en attente. (Accès Public) .

- **GET /api/v1/inscriptions/{id}** : Récupération d'un dossier d'inscription spécifique par son identifiant unique. (Accès Public) .
- **PUT /api/v1/inscriptions/{id}/valider** : Endpoint critique permettant à un rôle ADMIN ou PROFESSOR de valider formellement un dossier. **Sécurisé obligatoirement par jeton MicroProfile JWT**.

4. Sécurité Avancée (MicroProfile JWT)

Pour garantir le modèle *Stateless* (sans état) de l'architecture distribuée, la sécurité de l'API UniRegis ne repose pas sur des sessions de serveurs, mais sur l'implémentation de la spécification **MicroProfile JWT**.

Les accès aux endpoints sensibles (comme la validation) exigent un jeton signé numériquement via l'algorithme de chiffrement asymétrique **RS256** (clé publique/privée) , assurant une authentification inviolable des rôles d'administration.

5. Infrastructure de Déploiement Orchestrée (Docker Compose)

Le projet est packagé et déployé de manière transparente grâce à une infrastructure multi-conteneurs définie sous **Docker Compose (v3.8)**:

- **uniregis-db (PostgreSQL 15 Alpine)** : Conteneur de base de données relationnelle écoutant sur le port local 5432. Base de données configurée sous le nom `uniregis_db` avec l'utilisateur `uniregis_user`.
- **uniregis-app (Payara Server 6 Full Profile)** : Serveur d'applications d'entreprise tournant sous **Java JDK 21**. Il expose le port de l'API REST (8080) et la console d'administration GlassFish (4848). Il gère le déploiement à chaud automatique (*Hot Deploy*) par montage du fichier `.war`. Ce conteneur dépend explicitement du démarrage de la base de données (`depends_on: postgres`).
- **uniregis-metrics (Prometheus v2.45.0)** : Conteneur de surveillance configuré sur le port 9090. Il dépend du démarrage de Payara (`depends_on: payara`).

6. Analyse du Monitoring et de l'Observabilité

C'est le point central de notre architecture de production. Le serveur Payara expose de manière native deux piliers fondamentaux de MicroProfile pour l'observabilité :

A. La Santé du Système (MicroProfile Health)

Exposé sur les points d'accès `/health/ready` et `/health/live`, ce module permet d'interroger directement l'état des composants internes. Dans notre code Java, nous avons écrit une méthode de vérification explicite nommée **testerConnexionBase()** encapsulée dans la classe `DatabaseReadinessCheck`.

Elle renvoie une métrique de type jauge binaire (gauge) nommée :

Extrait de code

```
vendor_microprofile_health_check_status{name="testerConnexionBase"}
```

- **Explication technique de l'écran vide lors de vos tests :** Si l'interface de Prometheus renvoie un résultat vide pour cette requête (comme sur tes captures d'écran 1 et 2), c'est parce que l'horodatage sélectionné sur votre tableau de bord (est figé dans le passé. Pour corriger cela devant le jury, il est impératif de cocher l'option **Use local time** et de cliquer sur **Execute** afin que Prometheus affiche en temps réel la valeur active **1** (indiquant que la connexion JDBC entre Payara et PostgreSQL est saine et pleinement établie).

B. Les Métriques de Performance (MicroProfile Metrics)

Prometheus envoie des requêtes HTTP GET régulières (Scraping) sur l'endpoint /metrics pour archiver l'activité. C'est ce mécanisme standardisé (OpenMetrics) qui permet de suivre des métriques avancées telles que :

- `memory_usedHeap_bytes` (Gestion de la mémoire vive).
- `system_cpu_load` (Charge CPU de l'infrastructure).
- `connection_pool_H2Pool_usedConnection_total` (Activité microtemporelle de l'accès aux données).

7. Conclusion et Perspectives

En répondant parfaitement au cahier des charges, le projet **UniRegis** démontre la viabilité technique d'une implémentation de pointe sous **Jakarta EE 10**. L'architecture offre une évolutivité (extension horizontale) immédiate grâce à l'isolation Docker.

Cette réalisation a permis de consolider des compétences avancées en ingénierie logicielle : développement d'API REST asynchrones, sécurisation décentralisée par clés asymétriques JWT et mise en œuvre d'une surveillance proactive d'infrastructure.